

FUNDAMENTOS DOS ALGORITMOS E DAS LINGUAGENS DE PROGRAMAÇÃO

Aula 1

INTRODUÇÃO AOS ALGORITMOS

Introdução aos algoritmos

Olá, estudante! Nesta videoaula, você verá os conceitos essenciais dos algoritmos, vendo os diferentes tipos e revelando aplicações práticas que permeiam nosso cotidiano.

Além disso, abordaremos assuntos relativos à introdução aos algoritmos como: conceitos, tipos de algoritmos e a aplicação deles em exemplos do mundo real, fundamental para sua compreensão.

Não perca a oportunidade de aprimorar seus conhecimentos e expandir sua compreensão sobre algoritmos. Vamos lá?

Ponto de Partida

Olá, estudante.

A partir deste momento, exploraremos juntos os princípios dos algoritmos, a base sólida de qualquer programador. Ao embarcar nesta jornada, desvendaremos os segredos por trás da construção de soluções eficientes para problemas computacionais.

No decorrer da disciplina serão abordados conteúdos que irão introduzir você ao mundo da programação. Vamos explorar os diversos tipos de algoritmos, entendendo suas características únicas e como cada um deles se encaixa perfeitamente em diferentes contextos.



Além disso, ao final desta Aula, você vai compreender teoricamente o que são algoritmos, e ainda, será capaz de visualizar como aplicá-los em situações do mundo real.

Atualmente o desenvolvimento de sistemas e aplicativos ligados a programação de computadores está presente na sociedade. O desenvolvimento do raciocínio lógico é fundamental para que o profissional se desenvolva em qualquer área do conhecimento que deseja atuar.

Diante disso, será apresentado o desenvolvimento de um algoritmo com uma tarefa bem comum: a preparação de uma xícara de café por meio de passos simples que tornarão possível esta atividade.

Convido você a iniciar a construção de uma base sólida para sua jornada no universo da programação. Preparado para essa imersão?

Vamos Começar!

O objetivo desta aula é apresentar a introdução à algoritmos, seus conceitos e definições, suas principais características, além de iniciar você estudante neste enorme cenário de tecnologia em que estamos inseridos.

Aqui, exploraremos as bases que sustentam a magia por trás dos programas de computador, os algoritmos. Antes de abordarmos os conceitos de lógica de programação, é importante entender o que nos aguarda nesta viagem.

Em um vasto campo, onde a lógica é a bússola e os algoritmos são as trilhas que podemos percorrer, esses algoritmos, como mapas, guiarão nosso raciocínio na resolução de problemas, construção de soluções e criação de programas eficientes.

Conceitos e introdução aos algoritmos

Iniciando este tópico, vamos começar apresentando um pouco dos conceitos que envolvem os algoritmos. Mas o que são Algoritmos?

Os algoritmos são sequências lógicas de passos ou instruções, representando um conjunto de operações bem definidas e ordenadas para resolver um problema específico. Esses procedimentos, essenciais no campo da ciência da computação, formam a base para o desenvolvimento de programas e soluções computacionais.

Em essência, um algoritmo é como uma receita que guia a execução de tarefas. Assim como um cozinheiro segue passos precisos para preparar um prato, um programador cria algoritmos para alcançar resultados desejados em um contexto computacional. Cada passo deve ser claro, preciso e capaz de ser executado de forma determinística.

A versatilidade dos algoritmos permite que eles se adaptem a uma variedade de domínios, desde tarefas simples até problemas complexos. Seja ordenando números, buscando informações em grandes conjuntos de dados ou controlando um robô, os algoritmos são a linguagem universal da computação, traduzindo lógica e raciocínio em ações concretas.

Ao compreender os algoritmos, desvendamos os segredos por trás da eficiência computacional, da resolução de problemas e do desenvolvimento de software. Em última análise, os algoritmos capacitam a transformação de ideias em ações, revelando-se como o cerne da inteligência computacional.

A Lógica, do ponto de vista filosófico é um conjunto de ações que determinam processos intelectuais que são condições gerais do conhecimento verdadeiro. Essa definição vem da lógica aristotélica (FERREIRA, 1999). Em outras palavras, podemos definir a lógica como um conjunto de tarefas organizadas e estruturadas com um propósito bem definido para solução de um problema.

No seu livro *Estudo Dirigido de Algoritmos*, Manzano; Oliveira (2012, p. 14) define a lógica como:

uma sequência coerente, regular e necessária de acontecimentos, de coisas e na área da informática é a forma pela qual assertivas, pressupostos e instruções são organizados em um algoritmo para implementação de um programa de computador.

Diante destes conceitos de lógica apresentados, partimos então para os algoritmos que nada mais são do que as trilhas sinalizadas através da bússola, ou seja, a nossa lógica.

Tipos de algoritmos

Diante destes conceitos de lógica apresentados, partimos para os algoritmos que são as trilhas sinalizadas através da bússola, ou seja, a nossa lógica.

Explicando melhor, um algoritmo é a solução de um problema ou são os passos necessários para a conclusão de uma tarefa. Por exemplo: Digamos que você tenha que trocar o pneu de um carro. Logicamente existe uma sequência de instruções que deverão ser seguidas para a conclusão dessa tarefa. Isso é um Algoritmo. Dessa maneira observe as instruções desse algoritmo no Quadro 1, a seguir:

1	Segurança e Preparação
	Estacione veículo em local seguro, desligue o motor e acione os sinais de emergência
2	Ferramentas Prontas
	Certifique-se de ter as ferramentas necessárias: chave de roda, macaco e estepe
3	Elevação do Veículo
	Use o macaco para levantar o veículo, seguindo as instruções do manual do carro
4	Troca Rápida
	Remova a roda danificada e substitua pelo estepe. Aperte os parafusos
5	Finalização
	Baixe o veículo com cuidado, aperte os parafusos novamente e verifique a pressão do pneu substituído

Observe no Quadro 1 que as instruções de 1 a 5 são claras quanto a realização da atividade da troca de pneu. Seguindo esses passos é possível atingir o objetivo que é o resultado do algoritmo.

Depois que o algoritmo está pronto, apenas precisamos segui-lo. Estamos certos se pensamos assim, porém devemos lembrar que para desenvolver esse algoritmo, esses passos para realizar essa ou qualquer outra tarefa, estamos exercitando nosso raciocínio lógico e nossa capacidade intelectual.

Por esse motivo, o desenvolvimento do raciocínio lógico é um dos objetivos e disciplinas ligadas a lógica de programação e aos algoritmos.

Ao adentrarmos no universo da programação, deparamo-nos com desafios que exigem soluções bem estruturadas. Para tal, é importante compreender as distintas formas de representação de algoritmos. Ao todo são 3 (três) formas, a saber:

- Narrativa.
- Fluxograma.
- Pseudolinguagem.

Vamos abordar cada um desses três pilares de forma mais detalhada.

Forma Narrativa

Em nossa jornada, deparamo-nos, inicialmente, com o algoritmo narrativo, uma narrativa textual que guia o profissional que vai desenvolver o algoritmo pelos passos necessários para alcançar a solução.

Nesse ponto, utilizamos uma linguagem próxima ao nosso cotidiano, descrevendo de maneira clara e sequencial as ações a serem executadas. Essa forma de representação, embora acessível, torna-se mais desafiadora à medida que lidamos com problemas complexos, em que a clareza verbal pode encontrar limitações. Isso acontece pelo problema de comunicação, pois as vezes a interpretação não é realizada corretamente.

O exemplo da troca de pneu exibido no Quadro 1 é definitivamente um algoritmo representado pela forma narrativa. Acompanhe este algoritmo com algumas variações sendo um exemplo deste primeiro pilar de representação de um algoritmo.

Preparação:

- Estacione o veículo em local seguro e plano.
- Desligue o motor e acione o freio de estacionamento.
- Pegue o macaco, chave de roda e o estepe do porta-malas.

Elevação do Veículo:

- Localize o ponto de elevação no chassi do veículo.
- Posicione o macaco sob o ponto de elevação.
- Gire a manivela do macaco para elevá-lo até que a roda saia do chão.

- Gire a manivela do macaco para elevá-lo até que a roda saia do chão.

Remoção da Roda Danificada:

- Utilize a chave de roda para afrouxar as porcas da roda.
- Remova a roda danificada.

Instalação do Estepe:

- Coloque o estepe no lugar da roda danificada.
- Rosqueie as porcas manualmente até que fiquem ajustadas.

Baixando o Veículo:

- Gire a manivela do macaco no sentido oposto para baixar o veículo.
- Aperte completamente as porcas com a chave de roda.
- Guarde as ferramentas no porta-malas.

Fluxograma

O fluxograma é uma ferramenta gráfica poderosa que traduz a lógica de programação em um mapa visual compreensível. Cada elemento gráfico em um fluxograma representa uma instrução específica, decisão ou processo, criando uma representação visual clara do algoritmo. Essa abordagem torna-se especialmente útil ao lidar com algoritmos complexos, permitindo uma análise detalhada de cada etapa (MANZANO; OLIVEIRA, 2012).

No fluxograma, diferentes formas são usadas para denotar diferentes elementos. O início e o fim do algoritmo são representados por elipses, enquanto retângulos são usados para indicar instruções ou processos.

Setas conectam essas formas, delineando o fluxo lógico do programa. Decisões são frequentemente representadas por losangos, permitindo bifurcações no caminho conforme as condições serão abordadas mais à frente nesta disciplina.

A natureza visual do fluxograma simplifica a compreensão, mesmo para quem não é proficiente em linguagens de programação. Ao seguir as setas e formas, é possível visualizar de forma intuitiva como o algoritmo trabalha. Essa clareza visual é especialmente valiosa durante o design e a depuração de algoritmos, pois oferece uma visão abrangente da lógica subjacente.

Na Figura 1, a seguir, é apresentado um Fluxograma simples, conhecido também como Diagrama de Blocos dentro dos conteúdos de algoritmos presentes na literatura.

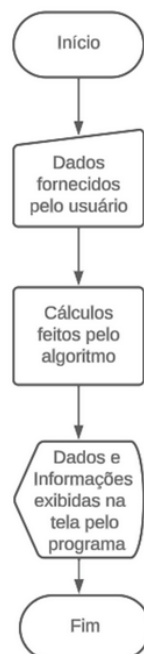


Figura 1 | Algoritmo para troca de pneu.

Em suma, o fluxograma é uma ferramenta essencial que pinta o caminho gráfico do raciocínio lógico por trás de um algoritmo.

Pseudolinguagem

À medida que nos aprofundamos em nossa exploração, deparamo-nos com a pseudolinguagem, uma forma de expressão que combina elementos da linguagem humana com estruturas de programação.

Essa abordagem busca ser uma ponte entre a compreensão intuitiva do algoritmo e sua posterior tradução para uma linguagem de programação específica. A pseudolinguagem oferece uma transição suave do conceitual para o prático, preparando o terreno para a codificação efetiva (MANZANO; OLIVEIRA, 2012).

Podemos dizer, também, que a pseudolinguagem é uma espécie de "linguagem de programação", porém ela não é considerada uma linguagem de programação por não ser utilizada profissionalmente. Neste sentido, ela é utilizada academicamente para o aprendizado, por ser de fácil compreensão. O "Portugol" é uma pseudolinguagem que permite o desenvolvimento de algoritmos estruturados em português de forma relativamente mais simples e intuitiva, o que já não acontece com uma linguagem de programação verdadeira.

Na Figura 2, apresentamos um exemplo simples na pseudolinguagem Portugol exemplificando como se programa nesta pseudolinguagem, ou seja, utilizando a forma de escrita: português estruturado.

```
1  Algoritmo ExemploPortugol
2  Var
3      caractere nome
4
5  Inicio
6      Escreva("Forneça seu nome.: ")
7      Leia(nome)
8
9      Escreva("Seu nome é.....: ", nome)
10
11     Escreva("Fim do Programa!")
12 Fim
```

Figura 2 | Pseudolinguagem "Portugol".

Essa representação em pseudolinguagem em Portugol oferece uma visão clara e legível do algoritmo, facilitando o entendimento das etapas lógicas envolvidas na entrada e saída de dados.



Siga em Frente...

Aplicação de algoritmos

Os algoritmos, embora muitas vezes associados à programação de computadores, são na verdade conjuntos de instruções lógicas utilizadas para resolver problemas e realizar tarefas específicas.

A aplicação de algoritmos permeia nosso cotidiano, sendo uma ferramenta fundamental em diversas áreas. Vamos explorar alguns contextos em que a lógica algorítmica desempenha um papel essencial.

- **Gerenciamento de listas e tarefas:** em nossas rotinas diárias, frequentemente utilizamos algoritmos para organizar tarefas e listas. Imagine um algoritmo simples que classifica automaticamente suas tarefas em ordem de prioridade, otimizando sua eficiência diária.
- **Sistemas de navegação GPS:** ao inserir um destino no GPS, o sistema utiliza algoritmos para calcular a rota mais rápida ou eficiente, considerando variáveis como distância, tráfego e preferências do usuário.
- **Redes sociais e recomendações:** plataformas de redes sociais aplicam algoritmos para personalizar *feeds*, exibindo conteúdo com base em interesses e interações anteriores. Algoritmos de recomendação também são usados para sugestões de amigos, vídeos ou produtos.
- **Processamento de transações financeiras:** no setor financeiro, algoritmos são indispensáveis para realizar operações como verificação de transações, detecção de fraudes e análise de padrões de gastos.
- **Medicina e diagnósticos:** algoritmos são empregados em sistemas de apoio ao diagnóstico médico, analisando dados clínicos para identificar padrões e sugerir possíveis tratamentos.
- **Filtros de spam em e-mails:** algoritmos são utilizados para analisar o conteúdo de e-mails e identificar padrões associados a mensagens indesejadas, contribuindo para a eficácia dos filtros de spam.
- **Classificação de imagens em aplicações fotográficas:** aplicações de reconhecimento de imagem usam algoritmos para classificar e organizar fotos com base em critérios como localização, rostos reconhecidos e data de captura.
- **Otimização de rotas de entrega:** empresas de logística utilizam algoritmos para otimizar rotas de entrega, minimizando custos e tempo, e garantindo uma distribuição eficiente.
- **Aprendizado de máquina e inteligência artificial:** algoritmos de aprendizado de máquina são considerados a espinha dorsal da inteligência artificial, capacitando sistemas a aprender e tomar decisões com base em dados.



A aplicação de algoritmos vai além da programação, alcançando áreas diversas de nossas vidas. Essas instruções lógicas fornecem soluções eficientes para uma ampla gama de desafios, desde a organização de tarefas diárias até o desenvolvimento de tecnologias avançadas. Ao compreender a importância dos algoritmos, somos capacitados a utilizá-los de maneira consciente e inovadora, contribuindo para uma sociedade cada vez mais conectada e eficiente.

Vamos Exercitar?

Descrição da situação-problema:

Seu desafio é criar um conjunto de passos simples para preparar uma xícara de café. Não precisamos de código por enquanto, apenas uma descrição narrativa do processo.

Resolução da situação-problema:

Verificação da Cafeteira:

- Antes de tudo, verifique se a cafeteira está pronta para uso.

Ligar a Cafeteira:

- Se a cafeteira estiver desligada, ligue-a.

Abastecer Água:

- Confira o nível de água na cafeteira.
- Se estiver baixo, adicione água até atingir a quantidade desejada.

Adicionar Café:

- Verifique a quantidade de café disponível.
- Se estiver acabando, adicione a quantidade necessária.

Preparar Café:

- Pressione o botão de preparo na cafeteira.
- Aguarde o processo de preparação ser concluído.

Servir Café:

- Com cuidado, despeje o café na xícara.

Apreciar:

- Por fim, sente-se, relaxe e aprecie sua xícara de café!

Esses passos representam um algoritmo simples para preparar café. Descrever o processo dessa maneira ajuda a entender a lógica por trás da tarefa antes de entrar em detalhes de programação. Este é apenas o início de sua jornada nos algoritmos.

Saiba Mais

Para saber mais sobre os conceitos e Introdução aos algoritmos no que diz respeito à formação de ideias para compreensão desse conteúdo, acesse em sua Biblioteca Virtual o livro: [Estudo Dirigido de Algoritmos](#). Capítulo 1, páginas 15 a 24.

Referências Bibliográficas

CORMEN, T. **Algoritmos - Teoria e Prática**. 3.ed. Rio de Janeiro: LTC, 2022.

CORMEN, T. **Desmistificando Algoritmos**. Grupo GEN, 2013.

FERREIRA, A. B. H. **Novo Aurélio Século XXI**: o dicionário da língua portuguesa. 3 ed. totalmente rev. e ampl. Rio de Janeiro: Nova Fronteira, 1999.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. **Estudo Dirigido de Algoritmos**. [S. l.]: Editora Saraiva, 2012.

MENÉNDEZ, A. **Simplificando Algoritmos**. Grupo GEN, 2023. E-book.

SERPA, M. S.; RODRIGUES, T. N.; ALVES, Í. C.; *et al.* **Análise de Algoritmos**. Grupo A, 2021.

CONCEITOS BÁSICOS DE LINGUAGENS DE PROGRAMAÇÃO

Conceitos básicos de linguagens de programação

Olá, estudante! Nesta videoaula, você verá os caminhos da sintaxe e semântica de linguagens de programação. Aprofundaremos ainda mais, explorando tipos de dados e desvendando os conceitos fundamentais das estruturas de controle básicas.

Viaje pelo universo dos dados, conhecendo suas variedades e entendendo como escolher o tipo certo de dado, pode potencializar suas soluções.

Você verá sobre estruturas de controle, aprendendo a dar direção e eficiência aos seus algoritmos.

Esteja pronto para essa imersão e não perca a oportunidade de aprimorar seus conhecimentos. Sua participação é indispensável para tornar essa experiência ainda mais enriquecedora.



Ponto de Partida

Olá, estudante! Nesta Aula, iniciaremos uma jornada conjunta na exploração dos conceitos básicos de linguagens de programação. Ao longo dessa Aula, vamos desvendar os mistérios para a criação de soluções eficazes em forma de algoritmos computacionais.

Serão apresentados conteúdos que o conduzirão ao vasto universo da programação. Vamos abordar a sintaxe e semântica de linguagens de programação e os possíveis tipos de dados utilizados no desenvolvimento de soluções algorítmicas.

Ao final desta Aula, você vai compreender alguns conceitos de estruturas de controle que serão utilizadas dentro do código-fonte.

Ligado ao desenvolvimento do raciocínio lógico, encontra-se a necessidade de estudar os elementos responsáveis por organizar esse raciocínio transformando em um algoritmo que solucione problemas propostos.

Diante disso, será apresentado a criação de um algoritmo para gerenciar informações sobre produtos comercializados por



Vamos lá?

Vamos Começar!

O objetivo desta aula é apresentar os conceitos básicos de linguagens de programação, compreendendo aspectos necessários que moldam a construção de algoritmos corretos e eficientes.

Ao compreender os conceitos da linguagem de programação, estaremos capacitados a expressar nossas ideias de maneira precisa e coerente em códigos compreensíveis.

Ao longo desta Aula, exploraremos os conteúdos que serão abordados com exemplos práticos e contextualizados. Prepare-se para absorver conhecimentos que não apenas enriquecerão seu repertório como programador, mas também abrirão as portas para a criação de soluções mais sofisticadas e eficazes.

Vamos desbravar os alicerces da programação, capacitando-o a construir algoritmos robustos e funcionais. Este é apenas o começo de uma jornada fascinante!

Sintaxe e semântica de linguagens de programação

A linguagem de programação é o veículo que nos permite comunicar instruções aos computadores, e seu entendimento é essencial para quem busca se aventurar no mundo da programação. Neste contexto, a sintaxe e semântica são pilares que fundamentam a construção de códigos coerentes e funcionais.

Iniciaremos nossa jornada mergulhando nesses conceitos de linguagens de programação. Imagine esses elementos como as regras gramaticais e o significado de uma linguagem, semelhantes à forma como palavras e frases constroem o entendimento em nossa comunicação cotidiana. Compreender essas estruturas é fundamental para expressar nossas ideias de forma clara e coesa no universo da programação.

Definições de sintaxe e semântica

A sintaxe refere-se às regras gramaticais e estruturais de uma linguagem de programação. É como a gramática de um idioma, determinando como os elementos devem ser organizados para criar instruções válidas.

Por exemplo, no português escrevemos "Sujeito + Verbo + Objeto," do mesmo modo, em uma linguagem de programação, seguimos regras específicas para criar comandos compreensíveis pela máquina (MENÉNDEZ, 2023).

A semântica, por sua vez, está relacionada ao significado das instruções. É como garantimos que, além de seguir as regras gramaticais, nossas instruções têm o significado desejado. Usando o mesmo exemplo, se em português dissermos "Planta + Água," sabemos que estamos falando sobre regar uma planta. Na programação, é importante que nosso código faça sentido e realize a ação esperada (MENÉNDEZ, 2023).

Analisando a sintaxe no idioma português, percebemos que seguimos estruturas específicas para formar frases. Verbos têm conjugações específicas para cada sujeito, e a ordem das palavras impacta o significado da frase. Em programação, a sintaxe segue uma lógica semelhante.

Palavras-chave, variáveis e operadores devem ser dispostos de maneira específica para que o código seja compreendido e executado corretamente.

Falando sobre o Portugol, que será nossa "pseudolinguagem" neste primeiro momento, vale lembrar que ele também é conhecido como português estruturado, é uma pseudolinguagem que possui como base a língua portuguesa.

A sintaxe no Portugol

O Portugol, uma linguagem pseudocódigo utilizada para ensinar programação que simplifica a sintaxe, e permite que o estudante compreenda os conceitos fundamentais antes de enfrentar linguagens mais complexas. Nela, os comandos são escritos de forma mais próxima do idioma humano, facilitando a transição do entendimento gramatical para a lógica de programação.

É por esse motivo que é recomendado primeiro o aprendizado do Portugol, ou seja, uma pseudolinguagem do que ir direto para uma linguagem de programação como C, Java ou Python.

Vamos explorar um exemplo prático na linguagem Portugol para reforçar esses conceitos. Suponha que desejamos criar um algoritmo que some dois números. Em Portugol, poderíamos ter algo como o apresentado na Figura 1.




```

1  ✓ Algoritmo Soma
2  ✓   Var
3      num1, num2, resultado: Real
4  ✓   Início
5       Escreva("Digite o primeiro número: ")
6       Leia(num1)
7
8       Escreva("Digite o segundo número: ")
9       Leia(num2)
10
11      resultado <- num1 + num2
12
13      Escreva("A soma é: ", resultado)
14      Fim

```

Figura 1 | Algoritmo "Soma".

Neste exemplo, a sintaxe é clara: temos comandos como "Escreva," "Leia," e o uso de operadores matemáticos. A semântica também é evidente, já que o código realiza a ação esperada, que é somar dois números.

Compreender a sintaxe e semântica é o alicerce para criar programas robustos. À medida que avançamos nos estudos, a clareza na escrita do código se torna essencial. Assim como em um idioma, praticar a construção de frases coesas é crucial para nos expressarmos eficientemente.

Para o entendimento completo do exemplo abordado na Figura 1, é necessário além de entender a sintaxe e semântica da escrita, outros conteúdos como por exemplo o conceito de tipos de dados.

Siga em Frente...

Tipos de dados

Os tipos de dados são a base fundamental para representar e manipular informações em um programa. Eles definem o conjunto de valores que uma variável pode armazenar, proporcionando uma riqueza de expressão aos nossos algoritmos.

Vejamos os quatro principais tipos de dados: **inteiros**, **reais**, **caracteres** e **lógicos**, compreendendo suas peculiaridades e explorando como são utilizados na linguagem de programação.

Inteiros

Os dados do tipo inteiro representam números inteiros, ou seja, valores sem parte fracionária. Em muitas linguagens de programação, como também na pseudolinguagem Portugol, os inteiros podem ser positivos ou negativos.

Por exemplo: os valores (0, -15, 4185, -3760) são exemplos de valores classificados como inteiros. Costumamos dizer também que o tipo de dado define qual a informação possível que será trabalhada, ou seja, quando definimos um objeto qualquer como "inteiro", estamos limitando a somente esse tipo de valor. Números sem casas decimais como exemplificado.

Reais

Os dados do tipo real representam números inteiros, e números fracionários, ou seja, valores numéricos de qualquer tipo. Podemos fazer uma comparação dizendo que os tipos reais atendem o que os tipos inteiros atendem e ainda são capazes de trabalhar com números fracionários. Se fosse uma concorrência de competências, os tipos reais certamente levariam a melhor.

Assim, se os tipos reais são uma "melhoria" dos tipos inteiros, pois atendem suas especificações e ainda com a possibilidade de trabalhar com números fracionários, por que usar os tipos inteiros? Vamos ficar somente com os tipos reais, certo? Errado, pois os tipos inteiros utilizam menos memória que os tipos reais.

Portanto, sempre que pudermos utilizar tipos inteiros, devemos fazê-lo. Um exemplo clássico é a idade de uma pessoa. Será geralmente um valor inteiro. Quando este for o caso, use o tipo de dado inteiro.

Exemplos de valores dos tipos reais são: (0, -15.3, 418.6, -7.509). Dentro dos algoritmos é muito comum o processamento de cálculos matemáticos. Quando o seu programa utilizar a operação de divisão e em outros casos de cálculos mais complexos, você vai precisar utilizar tipos reais no seu algoritmo.

Caracteres

Caracteres

Para representar palavras, textos ou símbolos especiais ou ainda números em conjunto com informações textuais, utilizamos o tipo de dado caractere.

Por exemplo, para trabalhar com a informação do endereço de uma pessoa, teremos valores textuais em conjunto com números, **"Rua Tecnologia da Informação, 5000"**. Repare que essa informação de um endereço qualquer tem o logradouro, o endereço e o número, ou seja, informações de vários tipos que serão representadas pelo tipo: caractere.

Essa categoria inclui letras, números, símbolos e espaços. A forma de representação sempre será dentro de aspas duplas " ", como representado no parágrafo anterior.

Explicando melhor, se o número **5000** aparecer sozinho, ele pode ser considerado um valor inteiro, mas se aparecer entre as aspas **"5000"**, ele passa a ser caractere. Outra restrição para o número 5000, agora sendo tratado como caractere é a impossibilidade de operações matemáticas, pois agora ele não é um valor numérico e sim um valor textual.

Lógicos

O tipo de dado lógico lida com valores de verdadeiro ou falso. Eles são importantes dentro do desenvolvimento de algoritmos mais complexos. Assim, os valores lógicos conseguem trabalhar somente com 2 valores: VERDADEIRO ou FALSO.

Para uma linguagem de programação, trabalhada no idioma inglês, esses valores serão TRUE ou FALSE. Eles são utilizados para armazenar o retorno de uma função, por exemplo.

Em Portugal, a manipulação desses tipos de dados é intuitiva. Podemos realizar operações matemáticas, comparações e concatenações de caracteres de forma simples. A linguagem facilita o entendimento dos conceitos antes de lidar com sintaxes mais complexas.

A seguir, na Tabela 1, é apresentada uma tabela com um exemplo de valores para cada tipo de dado.

Tipo de dado	Exemplo
Inteiro	42
Real	3.14
Caractere	"A"
Lógico	Verdadeiro

Tabela 1 | Exemplo tipos de dados.

Essa Tabela apresenta exemplos representativos de cada tipo de dado. Os inteiros e reais são utilizados para números, o caractere para representar letras ou símbolos, e o lógico para valores verdadeiro ou falso. Esses exemplos ajudam a compreender como os diferentes tipos de dados podem ser aplicados em programação.

Dominar os tipos de dados é uma jornada essencial na construção do conhecimento em programação. Cada tipo possui sua função única, e a escolha adequada é crucial para o sucesso do algoritmo.

Execução de um programa de computador

De acordo com Manzano; Oliveira (2012), os algoritmos criados por desenvolvedores devem ser convertidos para uma linguagem de alto nível, como: C, Java, etc. Devemos converter o algoritmo escrito para uma dessas linguagens de alto nível para que eles se tornem executáveis, mas cada linguagem possui um método único para realizar este processo.

De acordo com Manzano; Oliveira (2012), existem três métodos para gerar um código executável: Compiladores, Interpretadores e Tradutores.

Uma advogada, um estudante, um professor e diversos outros profissionais que utilizam computadores em suas atividades normalmente recorrem a uma ferramenta chamada processador de textos. Contadores, engenheiros, economistas, físicos, químicos, matemáticos e outros profissionais que lidam com cálculos fazem uso de uma ferramenta denominada planilha eletrônica ou por meio da nuvem utilizando serviços de editores compartilhados.

Já publicitários, desenhistas e profissionais de comunicação utilizam ferramentas gráficas, como programas de apresentação ou editores gráficos de desenho e fotografia. Na área de desenvolvimento de sistemas, os desenvolvedores de software também têm suas próprias ferramentas de trabalho, como editores de texto, compiladores e tradutores.

Como citado, ao concluir o projeto de um programa de computador, é necessário transformá-lo em software. Para isso, o projeto definido é traduzido para uma linguagem de programação formal, que é executada em um computador. Essa tarefa

envolve escrever o código do programa em uma ferramenta de edição de textos e, em seguida, submeter o programa a ferramentas de tradução, interpretação e compilação, conforme necessário.

O editor de texto, ferramenta utilizada para editar o texto de um programa, é um programa simples que permite escrever o código do programa e, basicamente, gravá-lo. Essa ferramenta possui recursos essenciais, como a capacidade de copiar e colar blocos de texto, salvar e recuperar arquivos e imprimi-los.

Exemplos de editores de texto incluem o EDIT no MS-DOS, o Notepad do Windows, conhecido como bloco de notas, e o VI (vi-ai) no Linux e UNIX. Não é necessário que um editor de texto ofereça mais recursos do que esses, pois, quando isso acontece, o programa deixa de ser um editor e passa a ser um processador de texto, como o "WordPad" ou "Microsoft Word" no Windows ou outros similares em outros sistemas operacionais.

As ferramentas de tradução são programas que possibilitam a conversão de um código em uma linguagem formal para outra linguagem também formal. Suponha que um programador, embora seja proficiente em programação e tenha uma lógica excelente, só consiga escrever programas na linguagem Java, e necessite entregar um programa-fonte em Python. O desafio é que ele não tem conhecimento da linguagem Python.

Nessa situação, ele pode recorrer a uma ferramenta de tradução. O programador escreve o programa-fonte em Java, e a ferramenta de tradução reescreve o código na linguagem Python, gerando um programa-fonte como se o próprio programador tivesse escrito. Há diversos tipos de tradutores disponíveis, tanto ferramentas de distribuição gratuita, como o programa p2c, quanto ferramentas comerciais.

As ferramentas de interpretação são programas que executam um programa-fonte diretamente na memória principal do computador, sem a necessidade de ser executado no processador central da máquina. Geralmente, esse tipo de ferramenta permite uma execução rápida dos programas. Diversas linguagens de programação, como Basic, Perl, Python, Forth, JavaScript e LOGO, adotam a interpretação como método de execução.

Já as ferramentas de compilação são programas que convertem um programa-fonte escrito em uma linguagem de alto nível para uma linguagem de baixo nível (linguagem de máquina).

Durante essa tradução, o programa-fonte transforma-se em um programa-objeto compatível com o processador em uso e, posteriormente, ocorre a ligação com as rotinas de execução do sistema operacional, transformando-o em um código executável. Embora os programas compilados possam apresentar um desempenho ligeiramente mais lento se comparados aos programas interpretados, eles asseguram a inacessibilidade ao código-fonte.

A escolha entre ferramentas de tradução, interpretação ou compilação, bem como a linguagem de programação a ser utilizada, depende de uma análise criteriosa e decisões a serem tomadas pela equipe de desenvolvimento alinhada com a gestão da empresa.

Em conclusão, as ferramentas de tradução, interpretação e compilação desempenham papéis únicos no ciclo de vida do desenvolvimento de software. A escolha entre essas abordagens depende das necessidades específicas de um projeto, considerando fatores como desempenho, acessibilidade ao código-fonte e as características da linguagem de programação.

As ferramentas de tradução oferecem a flexibilidade de converter códigos entre linguagens formais, como exemplificado pelo programa p2c. Já as ferramentas de interpretação permitem uma execução rápida dos programas diretamente na memória, mas podem expor o código-fonte.

Por fim, as ferramentas de compilação garantem a transformação do código-fonte em um programa executável, proporcionando maior segurança, embora com um potencial leve impacto no desempenho. A compreensão dessas ferramentas é essencial para os desenvolvedores, pois influencia diretamente na eficiência e na segurança das soluções computacionais que eles desenvolvem.

Portanto, a compreensão da sintaxe e semântica de linguagens de programação é essencial para construir códigos claros e funcionais. A exploração dos tipos de dados, incluindo inteiros, reais, caracteres e lógicos, proporciona ao programador a capacidade de manipular informações de forma eficiente.

Além disso, entendemos o processo de execução de um programa de computador, compreendendo as etapas de tradução, interpretação e compilação.

Todos esses conhecimentos são necessários para qualquer programador, constituindo os alicerces para a resolução de problemas complexos e o desenvolvimento de aplicações robustas e de alta performance.

Vamos Exercitar?

Descrição da situação-problema:



Em um ambiente de controle de estoque de uma loja de conveniência, surge a necessidade de desenvolver um algoritmo para gerenciar informações sobre os produtos disponíveis. O objetivo é garantir maior eficiência nas operações relacionadas ao estoque, como consulta e atualização de dados.

Descrição da Resolução:

Para solucionar essa demanda, é necessário criar um programa que contemple diferentes tipos de dados para representar as características dos produtos. Cada produto deve ser identificado por um código (inteiro), ter um preço unitário associado (real), uma descrição textual (caractere) e um indicador de disponibilidade em estoque (lógico).

Resolução da situação-problema:

Definição dos tipos de dados:

- Código: Inteiro.
- Preço Unitário: Real.
- Descrição: Caractere.
- Disponibilidade em Estoque: Lógico.

Registro de produtos:

- Cada produto será registrado com um código único, preço, descrição e disponibilidade inicial.

Consulta de produtos:

- O algoritmo deve permitir a consulta de informações de um produto específico, fornecendo seus detalhes.

Atualização de estoque:

- Operações para aumentar ou diminuir a quantidade de um produto em estoque, atualizando o indicador de disponibilidade.

Utilização do Algoritmo:

- O algoritmo será implementado em um sistema que permitirá aos funcionários gerenciar o estoque da loja de maneira eficiente.

Essa situação-problema introduz a necessidade de compreender e manipular tipos de dados, a saber: (inteiros, reais, caracteres e lógicos) para criar um algoritmo funcional no contexto do controle de estoque.

O desenvolvimento e teste do algoritmo proporcionarão *insights* sobre como esses tipos de dados são essenciais na resolução de problemas práticos.



Saiba Mais

Para saber mais sobre os conceitos e Introdução aos algoritmos no que diz respeito à formação de ideias para compreensão desse conteúdo, acesse em sua Biblioteca Virtual o livro: [Estudo Dirigido de Algoritmos](#). Capítulo 3 - "Programação com Sequência páginas 38 a 40.

Referências Bibliográficas

CORMEN, T. **Algoritmos - Teoria e Prática**. 3.ed. Rio de Janeiro: LTC, 2022.

CORMEN, T. **Desmistificando Algoritmos**. Grupo GEN, 2013.

FERREIRA, A. B. H. **Novo Aurélio Século XXI**: o dicionário da língua portuguesa. 3 ed. totalmente rev. e ampl. Rio de Janeiro: Nova Fronteira, 1999.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. **Estudo Dirigido de Algoritmos**. [S. l.]: Editora Saraiva, 2012.

MENÉNDEZ, A. **Simplificando Algoritmos**. Grupo GEN, 2023. E-book.

SERPA, M. S.; RODRIGUES, T. N.; ALVES, Í. C.; *et al.* **Análise de Algoritmos**. Grupo A, 2021.



COMPONENTES E ELEMENTOS DE LINGUAGEM DE PROGRAMAÇÃO

Componentes e elementos de linguagem de programação

Olá, estudante! Nesta videoaula, abordaremos temas como atribuições de valores, estrutura de um programa de computador e alguns fundamentos de variáveis e constantes.

Durante a Aula, desvendaremos os segredos por trás das operações básicas da matemática, e como essas operações são atribuídas a variáveis criando algoritmos consistentes. Além disso, exploraremos a estrutura essencial dos algoritmos e programas, proporcionando uma visão clara de como organizar suas instruções para alcançar soluções eficientes.

Estamos ansiosos para tê-lo conosco nesta jornada de aprendizado. Prepare-se para expandir seus horizontes na ciência da computação! Não perca essa oportunidade e confira a videoaula. Até lá!



Ponto de Partida

Olá, estudante!

Vamos abordar os fundamentos essenciais dos conteúdos dessa aula, para que no decorrer desta jornada você fique mais próximo do objetivo de dominar as técnicas dos algoritmos e da lógica de programação.

Ao abordar a "Introdução às operações e atribuições", veremos os princípios das ações executadas pelos algoritmos. Compreender como realizar operações e atribuições é essencial para construir lógicas eficientes na resolução de problemas computacionais.

A "Estrutura dos algoritmos e programas" será outro ponto focal. Vamos analisar como os algoritmos são organizados e estruturados para criar programas coesos e compreensíveis.

Além disso, exploraremos os "Fundamentos de variáveis e constantes". Variáveis e constantes são elementos-chave na programação, permitindo armazenar e manipular dados. Entender como utilizá-las corretamente é essencial para a criação de algoritmos eficazes.

Atualmente, o desenvolvimento de sistemas e aplicativos ligados a programação de computadores está muito presente na sociedade. Assim, o desenvolvimento do raciocínio lógico é fundamental para que o profissional se desenvolva em qualquer área do conhecimento que deseja atuar.

Diante disso, será apresentado o esboço de um algoritmo para automatizar uma cortina em uma residência por meio de



passos executados em uma sequência lógica que tornarão possível esta atividade.

Convido você a iniciar este conteúdo para continuar sua jornada no universo de aprendizado da programação. Vamos lá?

Vamos Começar!

Olá, estudante! Nesta Aula, faremos uma introdução a operações e atribuições, explorando como manipular e modificar dados, realizar cálculos e executar ações fundamentais para a resolução de problemas computacionais. Compreenderemos como essas operações são essenciais na construção de algoritmos que não apenas funcionam, mas que também são eficientes e elegantes.

Entenderemos como criar fluxos de execução que transformam problemas complexos em passos claros e compreensíveis, uma habilidade crucial para qualquer programador que quer entrar no mercado de trabalho.

Não menos importante, mergulharemos nos fundamentos de variáveis e constantes, elementos-chave que conferem dinamismo e adaptabilidade aos nossos algoritmos. Compreenderemos como esses componentes são essenciais para armazenar e manipular dados, possibilitando a criação de programas com esses recursos.

Fundamentos de variáveis e constantes

Na vasta paisagem da programação, os fundamentos de variáveis e constantes se destacam como pilares essenciais, conferindo flexibilidade e controle à manipulação de dados nos algoritmos.

Vejamos, então, o que são variáveis, e como utilizá-las de maneira eficaz no contexto do Portugol. As variáveis serão utilizadas em todas as linguagens de programação, porém neste momento vamos nos limitar em nossa pseudolinguagem, o Portugol.

Em Portugol, uma variável é um espaço de armazenamento identificado por um nome simbólico, que representa um valor ou informação. Ao contrário das constantes, as variáveis podem ter seu valor alterado durante a execução do programa. Elas são como gavetas na memória, onde podemos guardar e modificar dados conforme necessário metaforicamente falando.

A memória do dispositivo que estamos utilizando funciona como se fossem repositórios, caixas onde podemos colocar as informações dentro delas. Elas estão relacionadas aos tipos de dados, onde em cada gaveta podemos colocar somente valores do mesmo tipo.

Pense, por exemplo, em duas gavetas de meias e camisetas. Na gaveta de meias você não pode colocar camisetas, somente meias. Da mesma forma quando você cria uma variável você define que dentro dela você só vai aceitar valores numéricos, ou seja, se você quiser armazenar o nome de uma pessoa isso não será possível, pois o nome não é um valor numérico, logo você precisa criar outra gaveta, ou seja, outra variável (MENÉNDEZ, 2023).

As variáveis têm características essenciais, como o tipo de dado que podem armazenar, seja ele inteiro, real, caractere ou lógico. Além disso, possuem um escopo, determinando em que no programa elas podem ser acessadas, e uma sintaxe específica para declaração, indicando ao Portugol a reserva de espaço na memória (MENÉNDEZ, 2023).

Variáveis são indispensáveis para armazenar informações dinâmicas, como resultados intermediários, dados do usuário ou estados temporários. Podem conter valores que variam ao longo da execução do programa, proporcionando adaptabilidade.

No Portugol, seus valores possíveis são diretamente relacionados ao tipo de dado especificado durante a declaração como já citado nesta aula. Se você cria uma variável do tipo caractere, significa que vai poder armazenar valores textuais dentro da mesma (MENÉNDEZ, 2023).

Quando uma variável é criada, o Portugol aloca um espaço na memória do computador para armazenar seu valor. É fundamental inicializar uma variável antes de utilizá-la para evitar comportamentos inesperados, pois seu valor inicial pode ser indefinido se não especificado explicitamente.

Desse modo, a inicialização é o ato de atribuir um valor inicial à variável no momento da declaração. Isso pode evitar que exista "sujeita" no local onde a variável está utilizando na memória, visto que a memória pode ser compartilhada por outros programas e aplicativos na máquina que estão sendo executados concomitantemente. Assim, quase todos os algoritmos existentes utilizam um conjunto de variáveis no seu interior, por isso é muito importante absorver bem este conceito (MENÉNDEZ, 2023).

Outra questão é compreender que a variável que está no algoritmo é passível de alteração, ou seja, o seu conteúdo é volátil, incerto e totalmente alterável (MENÉNDEZ, 2023).

A exemplo disso, imaginemos que cada variável que utilizamos é constituída de três partes: **nome**, **tipo** e **conteúdo**.

O nome é dado quando declaramos a variável, assim como o tipo de informação que a variável vai trabalhar daqui pra frente. O dado que será colocado dentro da variável, virá de um **comando de atribuição** dentro do algoritmo ou de uma **leitura** da própria variável. Importante também ter este conceito em mente. Outros conceitos serão abordados posteriormente. Neste momento o que precisamos compreender são essas três partes que constituem uma variável.

Para exemplificar, na Tabela 1 é apresentada um esquema de variável apresentando o seu "MODELO", constituído por essas três partes e um exemplo de uma variável já "CRIADA".

Modelo de uma variável	Uma variável qualquer
Nome	Idade
Tipo de Dado	Inteiro
Valor	19

Tabela 1 | Esquema de uma variável.

Neste universo intrigante de variáveis e constantes em Portugol, compreendemos que elas não são apenas recipientes de dados, mas componentes dinâmicos que dão vida e flexibilidade aos nossos algoritmos.

Ao entender suas características, utilização e importância da inicialização, capacitamos nossos programas a lidarem de forma eficaz com a complexidade dos dados e das operações. Este conhecimento é essencial para qualquer programador que busca construir soluções robustas e adaptáveis.

Introdução as operações e atribuições

Na jornada da programação, a habilidade de realizar operações e atribuições é a essência da manipulação de dados em algoritmos. Vamos nos aprofundar nesse território fundamental, começando pela compreensão dos operadores de atribuição no contexto do Portugol.

Operadores de atribuição

Em Portugol, os operadores de atribuição são fundamentais para a manipulação de variáveis. O mais comum é o operador $\leftarrow$ (seta pra esquerda), utilizado para atribuir um valor a uma variável. Por exemplo, `idade  $\leftarrow$  25` atribui o valor 25 à variável `idade`. Esses operadores são como ferramentas que permitem moldar e modificar o comportamento do programa.

Os operadores de atribuição não apenas designam valores a variáveis, mas também podem ser combinados com outros operadores, como os operadores aritméticos por exemplo: (+, -, *, /), proporcionando uma forma concisa de realizar operações e atribuições simultaneamente. Por exemplo, `idade  $\leftarrow$  idade + 1` aumenta o valor da variável `idade` em 1.

Esses operadores são amplamente utilizados em situações em que é necessário atualizar, calcular ou modificar o valor de variáveis durante a execução do programa. Seja em um simples incremento numérico ou em operações mais complexas, os operadores de atribuição são ferramentas indispensáveis para o desenvolvedor.

Alteração de valores

A alteração de valores de uma variável pode ocorrer de duas formas em Portugol. A primeira é por meio do comando de leitura, onde o usuário fornece um valor que é então atribuído à variável. A segunda forma é pelo operador de atribuição, onde podemos diretamente especificar o valor a ser atribuído à variável.

Exemplificando melhor, o comando de leitura de dados é utilizado toda vez que é necessário capturar informações externas ao programa. Por exemplo: um algoritmo que solicite ao usuário sua senha. Este valor é uma entrada de dados externo que será utilizado na verificação dentro do algoritmo. A Figura 2 apresenta um código em Portugol mostrando este processo. Observe.

```
1  Algoritmo Senha
2  Var
3      nome, senha: caractere
4  Início
```

```

5      Escreva("Digite o seu nome...: ")
6      Leia(nome)
7
8      Escreva("Entre com sua senha.: ")
9      Leia(senha)
10
11     Escreva("Fim do Programa!!!")
12     Fim

```

Figura 2 | Comando de leitura de dados.

Como pode ser observado na Figura 2, a linha 9 do algoritmo "Senha", o comando Leia() faz a gravação do valor digitado quando o algoritmo está sendo executado na variável "senha". A partir desse momento o valor da variável é alterado.

A Figura 3, também apresenta a outra forma de mudança do valor da variável por meio do operador de atribuição, observe.

```

1  Algoritmo Senha
2  Var
3  |   nome, senha: caractere
4  Início
5  |   Escreva("Digite o seu nome...: ")
6  |   Leia(nome)
7
8  |   Leia ← "****45JW"
9
10 |   Escreva("Fim do Programa!!!")
11 Fim

```

Figura 3 | Operador de atribuição.

Percebemos que o algoritmo foi alterado para que o valor da variável senha seja alterado pelo operador de atribuição <- (seta para esquerda).

Ao entender como manipular variáveis por meio de operadores de atribuição, o programador compreende umas das funções mais utilizadas no desenvolvimento de algoritmos. A capacidade de manipular variáveis por meio de atribuições não é apenas um detalhe técnico; é a chave para desbloquear o potencial de criação e inovação no mundo da programação.

Siga em Frente...

Estrutura dos algoritmos e programas

No mundo da programação, a estrutura dos algoritmos e programas é um conceito fundamental que permeia toda a disciplina. Assim como a arquitetura de um edifício proporciona a base para sua funcionalidade, a estrutura dos algoritmos determina como as instruções são organizadas e executadas dentro de um programa de computador.

Esta é a essência que dá forma à lógica por trás do código, proporcionando uma compreensão clara e eficiente do funcionamento do software.

Explorando o universo da programação por meio do Portugol, uma linguagem simplificada e didática, somos introduzidos às estruturas fundamentais que constituem qualquer algoritmo.

A sequência, nossa primeira estaca, organiza as instruções em uma ordem linear, criando uma narrativa coerente. Em paralelo, o comando de atribuição entra em cena, permitindo-nos definir valores a variáveis e, assim, moldar dinamicamente o comportamento do nosso algoritmo.

Construindo com comandos simples, neste estágio inicial, optamos por comandos mais comuns, afastando-nos temporariamente das complexidades das estruturas condicionais, de repetição ou funções. Essa abordagem minimalista nos fornece uma visão clara da essência da programação, permitindo-nos focar na estruturação básica dos nossos algoritmos.

Portanto, cada linha de código, cada sequência de comandos, torna-se uma pedra preciosa na construção do conhecimento, preparando-nos para desafios mais avançados que virão à medida que avançamos nessa incrível jornada.

A estrutura dos algoritmos e programas desempenha um papel fundamental na programação, determinando como as instruções são organizadas e executadas. Imaginemos essa estrutura como a arquitetura de uma construção, onde cada elemento tem um lugar específico e uma função clara. No contexto da programação, a organização adequada é crucial para garantir que o código seja compreensível e eficiente.

Dentro do Portugol, as estruturas fundamentais incluem a sequência, que organiza as instruções em uma ordem linear, e o comando de atribuição, que define valores a variáveis. Essa simplicidade é essencial para construir algoritmos básicos sem a complexidade de estruturas condicionais, de repetição ou funções.

A Figura 4 apresenta o exemplo em Portugol. Observe.

```
1  algoritmo "Estrutura_Basica"
2  ✓ var
3      numero: inteiro
4  ✓ inicio
5      // Sequência de comandos
6      escreva("Digite um número: ")
7      leia(numero)
8
9      // Comando de atribuição
10     numero ← numero + 1
11
12     // Saída
13     escreva("O número incrementado é: ", numero)
14
15 fimalgoritmo
```

Figura 4 | Estrutura em Portugol.

A Figura 4 aborda uma estrutura simples em Portugol contendo a definição do nome do algoritmo, uma variável do tipo inteiro declarada, e o algoritmo solicitando o valor ao usuário e incrementando o valor lido somado de mais um valor fixo (1). Ele apresenta uma informação com o valor da variável.

A Figura 5 apresenta o exemplo em Java. Observe que este algoritmo solicita primeiro o valor para o usuário e somente na próxima linha cria a variável já atribuindo o valor lido para a mesma.

```
1  // Estrutura sequencial e comando de atribuição em Java
2  public class EstruturaBasica {
3      public static void main(String[] args) {
4          // Sequência de comandos
5          System.out.print("Digite um número: ");
6
7          // Comando de atribuição
8          int numero = Integer.parseInt(System.console().readLine());
9
10         // Saída
11         System.out.println("O número incrementado é: " + (numero + 1));
12     }
13 }
```

Figura 5 | Estrutura em Java.

A Figura 6 apresenta o exemplo em Python. Observamos que este algoritmo solicita o valor de um número ao usuário ao mesmo tempo que cria a variável que armazena o valor lido. Logo após, incrementa-se à variável o valor de 1 e posteriormente é exibida a mensagem do valor da variável na tela do programa.

```
1  # Estrutura sequencial e comando de atribuição em Python
2  # Sequência de comandos
3  numero = int(input("Digite um número: "))
4
5  # Comando de atribuição
6  numero = numero + 1
7
8  # Saída
9  print("O número incrementado é:", numero)
10
```

Figura 6 | Estrutura em Python.

Em conclusão, mesmo algoritmos simples e estruturas básicas são essenciais para compreender os princípios fundamentais da programação. O aprendizado dessa base permite a você, estudante, desenvolver habilidades que serão aplicadas em algoritmos mais complexos, tornando-se a fundação para um aprendizado continuado.

Vamos Exercitar?

Descrição da situação-problema:

Em um projeto de automação residencial, surge a necessidade de criar um programa para controlar a abertura e fechamento automático de cortinas em uma casa. O objetivo é facilitar o gerenciamento das cortinas, permitindo que os usuários programem horários específicos para que elas abram ou fechem automaticamente. O desenvolvimento desse sistema de automação será realizado utilizando a pseudolinguagem Portugol.